A pod is broken. The fix is slow, manual, and repetitive

- ✕ Every incident is the same dance: `get pods` → `describe` → `logs` → guess the cause.
- ✕ The failures rhyme — **CrashLoopBackOff, ImagePullBackOff, OOMKilled, Unschedulable** — but you re-diagnose from scratch each time.
- ✕ Deep expertise lives with **a few senior engineers**; everyone else is blocked or pages them at 2am.
- ✕ Under pressure people **fix the symptom** (restart it) not the cause — and it breaks again.

> **The gap:** the *reading and reasoning* is slow and mechanical — but the *deciding* must stay with a human. We want to automate the first without giving up the second.

— 02 · PROPOSED SOLUTION

A LangGraph agent that diagnoses — and asks before it acts

A **Kubernetes Troubleshooting Agent** built as a **LangGraph** state machine: point it at a namespace, it *investigates* a broken pod live — calling read-only tools in a ReAct loop — then gpt-4o returns the real root cause with the evidence, and it waits for a human to approve any fix.

01
Investigate
ReAct loop: the agent calls `describe` / `logs` tools itself.

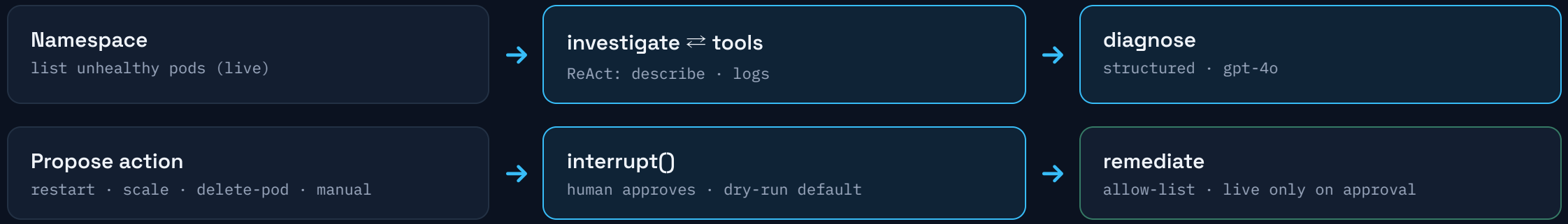
02
Diagnose
Structured output — failure class, evidence line, root cause, fix.

03
Propose
One safe action — restart / scale / delete-pod — or "manual".

04
Gate
Graph pauses at `interrupt()` for human approval.

The agent proposes; a human disposes. Real *live cases* on a real cluster — no canned data. The LLM only reads and reasons; the one mutating step is a tiny allow-list behind an explicit approval.

The process, end to end



Built with LangGraph: `StateGraph` + `ToolNode` + `interrupt()` for human-in-the-loop. Connects to OpenAI (gpt-4o) and your cluster (kubeconfig, read-only). Load real failures with `setup-cluster.sh`, then `streamlit run app.py`.